

大地坐标系和平面直角坐标系相互转换报告

一、运行方式

运行环境：VS2022 C#

可运行程序文件：大地坐标系和平面直角坐标系相互转换代码\bin\Debug\ DaDiToPingMian.exe

执行方式：

1. 选择大地基准和转换方式；
2. 输入待转换的坐标点；
3. 得到转换结果。

二、原理

大地坐标系基于地球椭球，用 大地纬度 B （过点的椭球法线与赤道面夹角）、大地经度 L （过点的子午面与起始子午面夹角）、大地高 H （点沿椭球法线到椭球面的距离）描述点的位置；空间直角坐标系以椭球中心为原点， X 轴指向起始子午面与赤道面交点， Y 轴在赤道面内垂直 X 轴， Z 轴沿椭球短轴（地球自转轴），用 (X, Y, Z) 描述点的位置。转换基于地球椭球的几何关系，椭球基本参

数：长半轴 a 、短半轴 b ，第一偏心率 $e = \sqrt{\frac{a^2 - b^2}{a^2}}$ 第二偏心率 $e' =$

$\sqrt{\frac{a^2 - b^2}{b^2}}$ ，卯酉圈曲率半径 $N = \frac{a}{\sqrt{1 - e^2 \sin^2 B}}$ 。

依据椭球面上点的几何关系，子午面内，椭球面上点的坐标满

足椭球方程，结合大地经纬度定义，可得：

$$\begin{cases} X = N \cos B \cos L \\ Y = N \cos B \sin L \\ Z = N(1 - e^2) \sin B \end{cases}$$

其中， N 是卯酉圈曲率半径，反映椭球在该点的横向弯曲程度，公式体现了大地经纬度 (B, L) 与空间直角坐标 (X, Y, Z) 的几何映射，通过椭球参数 N 、三角函数关系 ($\cos B$ 、 $\sin B$ 、 $\cos L$ 、 $\sin L$) 实现转换。

若点 P 大地高为 H ，其在椭球面上投影为 P_0 ，矢量关系为 $\rho = \rho_0 + H \cdot n$ (ρ 是 P 的空间位置矢量， ρ_0 是 P_0 的位置矢量， n 是椭球在 P_0 处外法线单位矢量)。推导得 P 点空间直角坐标：

$$\begin{cases} X = (N + H) \cos B \cos L \\ Y = (N + H) \cos B \sin L \\ Z = [N(1 - e^2) + H] \sin B \end{cases}$$

由空间直角坐标 (X, Y)，利用平面直角坐标与极角的关系，大地经度 L 是 XY 平面内点相对于 X 轴（起始子午面）的方位角，公式： $L = \arctan\left(\frac{Y}{X}\right)$ 。也可通过反正弦、反余弦表示： $L = \arcsin\left(\frac{Y}{\sqrt{X^2+Y^2}}\right)$ 或 $L = \arccos\left(\frac{X}{\sqrt{X^2+Y^2}}\right)$ ，本质是利用三角函数定义，从空间直角坐标的 (X, Y) 分量提取子午面相对位置信息。

因大地纬度 B 与椭球几何关系复杂，需迭代求解。设 $OP'' = \sqrt{X^2 + Y^2}$ (P'' 是 P 在赤道面投影)，结合椭球参数，利用三角关系可得初始迭代公式：

$$\tan B = \frac{Z + Ne^2 \sin B}{\sqrt{X^2 + Y^2}} \text{ 或 } \cot B = \frac{\sqrt{X^2 + Y^2} - Ne^2 \cos B}{Z}$$

由于公式右端含待求 B ，需迭代计算。初始值取 $\tan B_1 = \frac{Z}{\sqrt{X^2 + Y^2}}$ ，用 B_1 计算 $N_1 = \frac{a}{\sqrt{1 - e^2 \sin^2 B_1}}$ ；迭代更新：代入公式计算新 B ，直至相邻两次 B 差值小于允许误差。为简化迭代，引入优化公式（利用 $N = \frac{c}{\sqrt{1 + e'^2 \cos^2 B}}$ ， $c = \frac{a^2}{b}$ 为极曲率半径），整理得：

$$\tan B = \frac{Z}{\sqrt{X^2 + Y^2}} + \frac{ce^2 \tan B}{\sqrt{X^2 + Y^2} \cdot \sqrt{1 + e'^2 + \tan^2 B}}$$

迭代形式 $t_{i+1} = t_0 + \frac{pt_i}{\sqrt{k + t_i^2}}$ ($t = \tan B$ ， $t_0 = \frac{Z}{\sqrt{X^2 + Y^2}}$ ， $p = \frac{ce^2}{\sqrt{X^2 + Y^2}}$ ， $k = 1 + e'^2$)，通过参数替换、公式变形，将复杂非线性关系转化为便于迭代收敛的形式，逐步逼近精确的大地纬度 B 。

当地纬度 B 确定后，利用几何关系推导： $H = \frac{Z}{\sin B} - N(1 - e^2)$ 或 $H = \frac{\sqrt{X^2 + Y^2}}{\cos B} - N$ 。原理是将空间直角坐标 Z 分量（或 $\sqrt{X^2 + Y^2}$ ）沿椭球法线方向分解，减去椭球面上对应点几何高度（由 N 及椭球参数决定），得到大地高 H 。

转换本质是利用地球椭球的几何模型，通过正算（大地坐标→空间直角坐标，基于椭球参数与三角函数直接映射）和反算（空间直角坐标→大地坐标，大地经度直接提取，大地纬度迭代求解、大地高几何分解），建立两种坐标系的数学映射关系，核心依赖椭球参数、几何矢量合成与三角函数关系，反算中大地纬度因涉及椭球曲率的非线性关系需迭代处理，是大地测量、导航定位等领域坐标转换的基础理论。

三、代码

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace DaDiToPingMian
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender, EventArgs e)
        {
            // 初始化下拉框
            comboBox1.Items.Clear(); // 清空现有项
            comboBox1.Items.AddRange(TName);
            comboBox1.SelectedIndex = 0; // 默认选择 WGS-84 椭球体
            // 初始化表格
            InitializeDataGridViews();
        }
        // 椭球体参数
        string[] TName = { "克拉索夫斯基椭球体", "1975 年国际椭球体", "WGS-84 椭球体", "2000 中国大地坐标系" };
        double[] EList = { 0.006693421622966, 0.006694384999588, 0.00669437999013, 0.00669438002290 };
        double[] EAList = { 0.006738525414683, 0.006739501819473, 0.00673949674227, 0.00673949677548 };
        double[] a = { 6378245, 6378140, 6378137, 6378137 };
        double[] b = { 6356698.9017827110, 6399596.6519880105, 6399593.6258, 6399593.6259 };
        bool status = false; // 转换状态: false-大地坐标转空间直角坐标, true-空间直角坐标转大地坐标
        double[,] dataList1, dataList2; // 输入数据和结果数据
        private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
        {
            int index = comboBox1.SelectedIndex;
            if (index >= 0 && index < TName.Length)
            {
                label1.Text = $"当前选择: \n 椭球: {TName[index]}\n\n 椭球参数:  

                \na={a[index]}\nb={b[index]}\ne²={EList[index]:F12}\ne²={EAList[index]:F12}";
            }
        }
    }
}
```

```

    }
}

private void button1_Click(object sender, EventArgs e)
{
    // 保存当前数据
    SaveCurrentData();
    // 切换状态
    status = !status;
    label2.Text = status ? "空间直角坐标 → 大地坐标" : "大地坐标 → 空间直角坐标";
    // 更新表格列结构（不清空数据）
    UpdateDataGridViews();
    // 重新加载数据
    LoadSavedData();
    // 确保上方表格始终为输入，下方为输出
    dataGridView1.Location = new Point(278, 49);
    dataGridView2.Location = new Point(278, 203);
}

private DataTable savedInputData = new DataTable();
private DataTable savedOutputData = new DataTable();
private void SaveCurrentData()
{
    // 保存当前输入表格数据
    savedInputData = new DataTable();
    foreach (DataGridViewColumn col in dataGridView1.Columns)
    {
        savedInputData.Columns.Add(col.Name, typeof(string));
    }
    foreach (DataGridViewRow row in dataGridView1.Rows)
    {
        if (!row.IsNewRow)
        {
            DataRow dataRow = savedInputData.NewRow();
            for (int i = 0; i < row.Cells.Count; i++)
            {
                dataRow[i] = row.Cells[i].Value?.ToString() ?? "";
            }
            savedInputData.Rows.Add(dataRow);
        }
    }
}

// 保存当前输出表格数据
savedOutputData = new DataTable();
foreach (DataGridViewColumn col in dataGridView2.Columns)
{
    savedOutputData.Columns.Add(col.Name, typeof(string));
}

```

```

    }
    foreach (DataGridViewRow row in dataGridView2.Rows)
    {
        if (!row.IsNewRow)
        {
            DataRow dataRow = savedOutputData.NewRow();
            for (int i = 0; i < row.Cells.Count; i++)
            {
                dataRow[i] = row.Cells[i].Value?.ToString() ?? "";
            }
            savedOutputData.Rows.Add(dataRow);
        }
    }
}

private void LoadSavedData()
{
    // 清空表格但保留列结构
    dataGridView1.Rows.Clear();
    dataGridView2.Rows.Clear();
    // 恢复输入表格数据
    if (savedInputData.Columns.Count > 0)
    {
        foreach (DataRow dataRow in savedInputData.Rows)
        {
            DataGridViewRow row = new DataGridViewRow();
            row.CreateCells(dataGridView1);
            for (int i = 0; i < dataRow.ItemArray.Length && i < dataGridView1.Columns.Count; i++)
            {
                row.Cells[i].Value = dataRow[i];
            }
            dataGridView1.Rows.Add(row);
        }
    }
    // 恢复输出表格数据
    if (savedOutputData.Columns.Count > 0)
    {
        foreach (DataRow dataRow in savedOutputData.Rows)
        {
            DataGridViewRow row = new DataGridViewRow();
            row.CreateCells(dataGridView2);
            for (int i = 0; i < dataRow.ItemArray.Length && i < dataGridView2.Columns.Count; i++)
            {
                row.Cells[i].Value = dataRow[i];
            }
        }
    }
}

```

```

        dataGridView2.Rows.Add(row);
    }
}

private void InitializeDataGridViews()
{
    // 清空表格列
    dataGridView1.Columns.Clear();
    dataGridView2.Columns.Clear();
    // 添加点号列
    dataGridView1.Columns.Add("PointID", "点号");
    dataGridView2.Columns.Add("PointID", "点号");
    if (status) // 空间直角坐标转大地坐标
    {
        // 配置输入表格列
        dataGridView1.Columns.Add("X", "X(m)");
        dataGridView1.Columns.Add("Y", "Y(m)");
        dataGridView1.Columns.Add("Z", "Z(m)");
        // 配置输出表格列
        dataGridView2.Columns.Add("B", "B(°)");
        dataGridView2.Columns.Add("L", "L(°)");
        dataGridView2.Columns.Add("H", "H(m)");
    }
    else // 大地坐标转空间直角坐标
    {
        // 配置输入表格列
        dataGridView1.Columns.Add("B", "B(°)");
        dataGridView1.Columns.Add("L", "L(°)");
        dataGridView1.Columns.Add("H", "H(m)");
        // 配置输出表格列
        dataGridView2.Columns.Add("X", "X(m)");
        dataGridView2.Columns.Add("Y", "Y(m)");
        dataGridView2.Columns.Add("Z", "Z(m)");
    }
    // 设置表格样式
    SetupDataGridViewStyles();
}

private void UpdateDataGridViews()
{
    // 保存当前列标题
    Dictionary<int, string> inputColumnHeaders = new Dictionary<int, string>();
    Dictionary<int, string> outputColumnHeaders = new Dictionary<int, string>();
    for (int i = 0; i < dataGridView1.Columns.Count; i++)
    {

```

```

        inputColumnHeaders[i] = dataGridView1.Columns[i].HeaderText;
    }
    for (int i = 0; i < dataGridView2.Columns.Count; i++)
    {
        outputColumnHeaders[i] = dataGridView2.Columns[i].HeaderText;
    }
    // 清空表格列
    dataGridView1.Columns.Clear();
    dataGridView2.Columns.Clear();
    // 添加点号列
    dataGridView1.Columns.Add("PointID", "点号");
    dataGridView2.Columns.Add("PointID", "点号");
    if (status) // 空间直角坐标转大地坐标
    {
        // 配置输入表格列
        dataGridView1.Columns.Add("X", "X(m)");
        dataGridView1.Columns.Add("Y", "Y(m)");
        dataGridView1.Columns.Add("Z", "Z(m)");
        // 配置输出表格列
        dataGridView2.Columns.Add("B", "B(°)");
        dataGridView2.Columns.Add("L", "L(°)");
        dataGridView2.Columns.Add("H", "H(m)");
    }
    else // 大地坐标转空间直角坐标
    {
        // 配置输入表格列
        dataGridView1.Columns.Add("B", "B(°)");
        dataGridView1.Columns.Add("L", "L(°)");
        dataGridView1.Columns.Add("H", "H(m)");
        // 配置输出表格列
        dataGridView2.Columns.Add("X", "X(m)");
        dataGridView2.Columns.Add("Y", "Y(m)");
        dataGridView2.Columns.Add("Z", "Z(m)");
    }
    // 设置表格样式
    SetupDataGridViewStyles();
}
private void SetupDataGridViewStyles()
{
    foreach (DataGridView grid in new[] { dataGridView1, dataGridView2 })
    {
        // 设置列宽自适应
        grid.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.AllCells;
        // 设置列样式

```

```

        foreach (DataGridViewColumn col in grid.Columns)
        {
            col.SortMode = DataGridViewColumnSortMode.NotSortable;
        }
        // 设置单元格样式
        grid.DefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleRight;
        grid.DefaultCellStyle.Format = "F8"; // 默认 8 位小数
    }
    // 为输入表格设置特定格式
    if (status) // 输入为空间直角坐标
    {
        dataGridView1.Columns[1].DefaultCellStyle.Format = "F3"; // X 坐标 3 位小数
        dataGridView1.Columns[2].DefaultCellStyle.Format = "F3"; // Y 坐标 3 位小数
        dataGridView1.Columns[3].DefaultCellStyle.Format = "F3"; // Z 坐标 3 位小数
    }
    else // 输入为大地坐标
    {
        dataGridView1.Columns[1].DefaultCellStyle.Format = "F8"; // B 坐标 8 位小数
        dataGridView1.Columns[2].DefaultCellStyle.Format = "F8"; // L 坐标 8 位小数
        dataGridView1.Columns[3].DefaultCellStyle.Format = "F3"; // H 坐标 3 位小数
    }
    // 设置输入表格允许添加行
    dataGridView1.AllowUserToAddRows = true;
    dataGridView1.AllowUserToDeleteRows = true;
    // 设置输出表格为只读
    dataGridView2.AllowUserToAddRows = false;
    dataGridView2.AllowUserToDeleteRows = false;
    dataGridView2.ReadOnly = true;
}

private void button2_Click(object sender, EventArgs e)
{
    try
    {
        // 检查是否选择了椭球体
        if (comboBox1.SelectedIndex < 0)
        {
            MessageBox.Show("请先选择椭球体!", "提示", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }
        // 确保上方表格为输入
        DataGridView inputGrid = dataGridView1;
        int selectedIndex = comboBox1.SelectedIndex;
        double ea2 = EAList[selectedIndex]; // 第二偏心率平方
    }
}

```

```

double e2 = EList[selectedIndex];    // 第一偏心率平方
double semiMajorAxis = a[selectedIndex]; // 长半轴
// 获取实际数据行数（排除新添加的空行）

int rows = inputGrid.Rows.Count;
if (inputGrid.AllowUserToAddRows)
    rows--; // 减去自动添加的空行
if (rows <= 0)
{
    MessageBox.Show("请先输入数据！", "提示",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}

int cols = inputGrid.Columns.Count;
// 初始化数据数组
dataList1 = new double[rows, cols];
dataList2 = new double[rows, cols];
// 从输入表格读取数据
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        if (i < inputGrid.Rows.Count && j < inputGrid.Columns.Count)
        {
            object cellValue = inputGrid.Rows[i].Cells[j].Value;
            if (cellValue == null || cellValue == DBNull.Value)
                dataList1[i, j] = 0;
            else if (double.TryParse(cellValue.ToString(), out double result))
                dataList1[i, j] = result;
            else
                dataList1[i, j] = 0;
        }
    }
}

// 进行坐标转换
for (int i = 0; i < rows; i++)
{
    if (status) // 空间直角坐标-> 大地坐标
    {
        double X = dataList1[i, 1];
        double Y = dataList1[i, 2];
        double Z = dataList1[i, 3];
        double p = Math.Sqrt(X * X + Y * Y);
        // 处理极点特殊情况
        if (p < 1e-10) // 接近极点

```

```

    {
        dataList2[i, 0] = dataList1[i, 0]; // 点号
        dataList2[i, 1] = Z >= 0 ? 90.0 : -90.0; // 纬度为±90°
        dataList2[i, 2] = 0.0; // 经度设为0（极点经度无意义）
        dataList2[i, 3] = Math.Abs(Z) - b[selectedIndex]; // 大地高
        continue;
    }
    // 迭代法计算纬度 B
    double B = Math.Atan2(Z, p); // 初始值
    double tolerance = 5e-10; // 收敛容差 (5×10-10)
    double delta = 1.0; // 迭代步长
    int maxIterations = 10; // 最大迭代次数
    int iteration = 0;
    while (delta > tolerance && iteration < maxIterations)
    {
        double sinB = Math.Sin(B);
        double cosB = Math.Cos(B);
        // 防止除零错误
        if (Math.Abs(cosB) < 1e-10)
            break;
        double N = semiMajorAxis / Math.Sqrt(1 - e2 * sinB * sinB);
        double B_next = Math.Atan2(Z, p * (1 - e2 * N / (N + Z / sinB)));
        delta = Math.Abs(B_next - B);
        B = B_next;
        iteration++;
    }
    double L = Math.Atan2(Y, X);
    double N_final = semiMajorAxis / Math.Sqrt(1 - e2 * Math.Pow(Math.Sin(B), 2));
    // 防止除零错误
    double cosB_final = Math.Cos(B);
    if (Math.Abs(cosB_final) < 1e-10)
        cosB_final = 1e-10; // 设置一个极小值，避免除零
    double H = p / cosB_final - N_final;
    // 确保结果数组索引有效
    if (i < dataList2.GetLength(0) && 0 < dataList2.GetLength(1))
        dataList2[i, 0] = dataList1[i, 0]; // 点号
    if (i < dataList2.GetLength(0) && 1 < dataList2.GetLength(1))
        dataList2[i, 1] = B * (180 / Math.PI); // 弧度转度
    if (i < dataList2.GetLength(0) && 2 < dataList2.GetLength(1))
        dataList2[i, 2] = L * (180 / Math.PI); // 弧度转度
    if (i < dataList2.GetLength(0) && 3 < dataList2.GetLength(1))
        dataList2[i, 3] = H;
    }
    else // 大地坐标-> 空间直角坐标

```

```

    {
        double B = dataList1[i, 1] * (Math.PI / 180); // 度转弧度
        double L = dataList1[i, 2] * (Math.PI / 180); // 度转弧度
        double H = dataList1[i, 3];
        double N = semiMajorAxis / Math.Sqrt(1 - e2 * Math.Pow(Math.Sin(B), 2));
        double X = (N + H) * Math.Cos(B) * Math.Cos(L);
        double Y = (N + H) * Math.Cos(B) * Math.Sin(L);
        double Z = (N * (1 - e2) + H) * Math.Sin(B);
        // 确保结果数组索引有效
        if (i < dataList2.GetLength(0) && 0 < dataList2.GetLength(1))
            dataList2[i, 0] = dataList1[i, 0]; // 点号
        if (i < dataList2.GetLength(0) && 1 < dataList2.GetLength(1))
            dataList2[i, 1] = X;
        if (i < dataList2.GetLength(0) && 2 < dataList2.GetLength(1))
            dataList2[i, 2] = Y;
        if (i < dataList2.GetLength(0) && 3 < dataList2.GetLength(1))
            dataList2[i, 3] = Z;
    }
}
// 将转换结果显示在下方表格中
dataGridView2.Rows.Clear();
for (int i = 0; i < rows; i++)
{
    if (i < dataList2.GetLength(0))
    {
        DataGridViewRow row = new DataGridViewRow();
        row.CreateCells(dataGridView2);
        for (int j = 0; j < cols && j < dataList2.GetLength(1); j++)
        {
            // 根据数据类型设置不同的显示格式
            if (j == 0) // 点号
                row.Cells[j].Value = dataList2[i, j];
            else if (j == 1 || j == 2) // 经纬度保留 8 位小数
                row.Cells[j].Value = dataList2[i, j].ToString("F8");
            else // 高度和坐标值保留 3 位小数
                row.Cells[j].Value = dataList2[i, j].ToString("F3");
        }
        dataGridView2.Rows.Add(row);
    }
}
MessageBox.Show("坐标转换完成！", "成功", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
catch (IndexOutOfRangeException ex)

```

```

{
    MessageBox.Show($"索引超出界限错误: {ex.Message}", "错误", MessageBoxButtons.OK, MessageBoxIcon.Error);
    Console.WriteLine($"堆栈跟踪: {ex.StackTrace}");
}
catch (Exception ex)
{
    MessageBox.Show($"发生错误: {ex.Message}", "错误", MessageBoxButtons.OK, MessageBoxIcon.Error);
    Console.WriteLine($"堆栈跟踪: {ex.StackTrace}");
}
}

private void dataGridView2_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    // 单元格点击事件
}

private void dataGridView1_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    // 单元格点击事件
}

private void button3_Click(object sender, EventArgs e) // 清空数据按钮事件
{
    dataGridView1.Rows.Clear();
    dataGridView2.Rows.Clear();
}
}
}

```

四、结果

初始界面:



选择椭球和转换方式：

大地坐标-空间直角坐标互转 - 张洋20232411

下拉菜单

克拉索夫斯基椭球体
克拉索夫斯基椭球体
1975年国际椭球体
WGS-84椭球体
2000中国大地坐标系

椭球参数:
a=6378245
b=6356698.90178271
e²=0.006693421623
e'²=0.006738525415

大地坐标 -> 空间直角坐标

反转

清空

开始转换

点号	B(°)	L(°)	H(m)
*			

点号	X(m)	Y(m)	Z(m)
----	------	------	------

大地坐标-空间直角坐标互转 - 张洋20232411

下拉菜单

WGS-84椭球体

当前选择:
椭球: WGS-84椭球体

椭球参数:
a=6378137
b=6399593.6258
e²=0.006694379990
e'²=0.006739496742

空间直角坐标 -> 大地坐标

反转

清空

开始转换

点号	X(m)	Y(m)	Z(m)
*			

点号	B(°)	L(°)	H(m)
----	------	------	------

转换示例：

大地坐标-空间直角坐标互转 - 张洋20232411

下拉菜单

WGS-84椭球体

当前选择:
椭球: WGS-84椭球体

椭球参数:
a=6378137
b=6399593.6258
e²=0.006694379990
e'²=0.006739496742

空间直角坐标 -> 大地坐标

反转

清空

开始转换

点号	X(m)	Y(m)	Z(m)
1	30	55	69
2	200	248	369
*			

点号	B(°)	L(°)	H(m)
1.00000000	47.95300872	61.38954033	-6389848.936
2.00000000	49.38272592	51.11550357	-6389984.399

成功

坐标转换完成!

确定